

Guia Completo de Docker

Do conceito à prática — para desenvolvedores e engenheiros de dados

Matheus · 2026

Este guia cobre Docker do zero: o problema que ele resolve, conceitos fundamentais, Dockerfile, Docker Compose, volumes, redes e aplicações práticas em projetos de dados e web.

Fase 1 — Fundamentos

O que é Docker e por que ele existe

Docker é uma plataforma que permite empacotar uma aplicação junto com todo o ambiente que ela precisa para rodar — bibliotecas, versões de linguagem, variáveis de ambiente — num pacote chamado **container**. O container roda de forma isolada e idêntica em qualquer máquina que tenha Docker instalado.

O problema que ele resolve

"Na minha máquina funciona" — esse é o problema clássico. Cada desenvolvedor tem uma versão diferente de Python, libs e configurações. Com Docker, o ambiente vai junto com o código. Todo mundo roda exatamente a mesma coisa.

Três problemas reais que Docker resolve

- **Ambiente inconsistente:** versões diferentes de libs causam bugs que aparecem só em produção.
- **Dependências conflitantes:** dois projetos precisam de versões diferentes do Python — sem Docker há conflito.
- **Serviços locais:** subir PostgreSQL, Redis etc. sem instalar nada permanente na máquina.

Conceitos fundamentais

Conceito	O que é	Analogia
Imagem	Arquivo estático e imutável. Receita do ambiente.	Como uma classe em POO
Container	Instância da imagem em execução. Processo isolado.	Como um objeto instanciado
Dockerfile	Arquivo de texto com instruções para criar uma imagem.	Como uma receita de bolo
Docker Compose	Orquestrador de múltiplos containers via arquivo .yaml.	Como um maestro de orquestra

Imagem vs Container

- **Imagem:** imutável, fica no disco, criada com docker build, compartilhada via Docker Hub.
- **Container:** processo rodando, tem rede e arquivos próprios, criado com docker run.
- Você pode ter vários containers rodando da mesma imagem simultaneamente.
- Dados dentro do container somem ao remover — use volumes para persistir.

Docker vs Máquina Virtual (VM)

A diferença fundamental está na arquitetura. Uma VM virtualiza o hardware inteiro e carrega um sistema operacional completo — por isso pesa GBs e demora minutos para subir. O Docker compartilha o kernel do sistema operacional host e isola apenas o processo da aplicação.

Característica	VM	Docker
Tamanho	GBs por instância	MBs por container
Tempo de boot	Minutos	Segundos
Isolamento	Total — SO próprio	Por processo — kernel compartilhado
Segurança	Mais forte	Boa, mas menor
Uso de recursos	Alto	Muito baixo
Portabilidade	Limitada	Alta
SO suportado nativamente	Qualquer	Linux (Docker Desktop usa VM leve no Mac/Win)

Na prática, Docker e VM coexistem: o Docker Desktop no Mac e Windows roda uma VM Linux leve por baixo dos panos para hospedar o Docker Engine. Você usa Docker no dia a dia e a VM fica invisível.

Comandos básicos da Fase 1

```
docker pull python:3.12 # baixar imagem do Docker Hub
docker run python:3.12 # criar e rodar container
docker ps # containers em execução
docker ps -a # todos os containers
docker stop <id> # parar container
docker rm <id> # remover container
docker images # imagens locais
```

Fase 2 — Dockerfile e Imagens

O que é um Dockerfile

Um Dockerfile é um arquivo de texto simples, sem extensão, com instruções sequenciais que o Docker executa de cima para baixo para construir uma imagem. Você escreve uma vez e o Docker reproduz o ambiente idêntico em qualquer lugar.

Instruções principais

Instrução	O que faz	Exemplo
FROM	Define a imagem base. Sempre a primeira instrução.	FROM python:3.12
WORKDIR	Define o diretório de trabalho dentro do container.	WORKDIR /app
COPY	Copia arquivos da sua máquina para dentro da imagem.	COPY . .
RUN	Executa comando durante o build — instala dependências.	RUN pip install -r requirements.txt
ENV	Define variáveis de ambiente dentro do container.	ENV PYTHONUNBUFFERED=1
EXPOSE	Documenta qual porta a aplicação usa (não abre de fato).	EXPOSE 8000
CMD	Comando padrão que roda quando o container sobe.	CMD ["python", "app.py"]

Sistema de layers e cache

Cada instrução do Dockerfile cria uma camada (layer) na imagem. O Docker cacheia essas camadas — se você mudar só o código, ele não reinstala as dependências. Mas atenção: quando uma camada invalida o cache, todas as camadas abaixo dela são reconstruídas.

Regra de ouro:

Coloque o que muda menos no topo, o que muda mais embaixo. Dependências mudam raramente — código muda o tempo todo. Copie o requirements.txt e instale as libs ANTES de copiar o restante do código.

Exemplo — ordem correta:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt . # copia só o requirements primeiro
RUN pip install -r requirements.txt # instala — fica cacheado
COPY . . # copia o código (muda com frequência)
CMD ["python", "app.py"]
```

Docker Hub

Registro público de imagens — funciona como um GitHub, mas para imagens Docker. Quando você faz `FROM python:3.12`, o Docker baixa essa imagem do Docker Hub automaticamente. Você também pode publicar suas próprias imagens lá, de forma pública ou privada.

```
docker build -t meu-projeto:1.0 . # construir imagem com tag
docker push usuario/meu-projeto:1.0 # publicar no Docker Hub
docker pull usuario/meu-projeto:1.0 # baixar em qualquer máquina
```

Volumes

Por padrão, dados criados dentro de um container somem quando ele é removido. Volumes são pastas do seu sistema montadas dentro do container — o que for gravado nelas persiste.

- **Banco de dados local:** dados do PostgreSQL ficam num volume, não somem ao parar o container.
- **Desenvolvimento:** monte seu código como volume — mudanças refletem dentro do container sem rebuild.
- **Compartilhar dados:** dois containers podem ler e gravar no mesmo volume.

```
# montando volume no docker run
docker run -v ./dados:/var/lib/postgresql/data postgres:15
```

Fase 3 — Docker Compose

O que é Docker Compose

Docker Compose permite definir e orquestrar múltiplos containers em um único arquivo `docker-compose.yml`. Com um único comando você sobe toda a infraestrutura — API, banco de dados, cache, storage — e eles se comunicam automaticamente numa rede interna.

Anatomia do `docker-compose.yml`

Chave	O que faz
<code>services:</code>	Seção principal. Cada entrada é um container gerenciado pelo Compose.
<code>image:</code>	Imagem pronta do Docker Hub a usar. Ex: <code>image: postgres:15</code>
<code>build:</code>	Aponta pro Dockerfile local. Ex: <code>build: .</code>
<code>ports:</code>	Mapeia porta do host para o container. Ex: <code>"8000:8000"</code>
<code>volumes:</code>	Monta volumes no container para persistir dados.
<code>depends_on:</code>	Define ordem de inicialização. Ex: API sobe depois do banco.
<code>environment:</code>	Variáveis de ambiente para o container.

Redes internas

O Compose cria automaticamente uma rede privada entre todos os serviços. Dentro dessa rede, cada container enxerga os outros pelo nome do serviço — não por IP. Se o banco se chama `db` no arquivo, a API se conecta usando `host="db"`.

Importante: `depends_on` garante que o container do banco sobe antes da API, mas não garante que o banco está pronto para aceitar conexões. Para isso use `healthcheck` em projetos mais robustos.

Variáveis de ambiente e `.env`

- O Compose lê automaticamente um arquivo `.env` no mesmo diretório.
- Use `${NOME_DA_VARIAVEL}` dentro do `docker-compose.yml` para referenciar as variáveis.
- O arquivo `.env` nunca vai para o Git — adicione ao `.gitignore`.
- Versione um `.env.example` com as chaves mas sem os valores.

Comandos essenciais do Compose

```
docker compose up # sobe todos os serviços
docker compose up -d # sobe em background (detached)
docker compose down # para e remove os containers
docker compose logs # vê logs de todos os serviços
docker compose logs api # logs de um serviço específico
docker compose ps # status dos serviços
docker compose build # reconstrói as imagens
```

Fase 4 — Aplicando nos seus projetos

Docker em projetos de dados

No Databricks o ambiente já é gerenciado pela plataforma — você não dockeriza notebooks do Databricks. O Docker entra antes disso: no desenvolvimento local, para testar lógica de pipeline, validar transformações e schemas sem gastar crédito de cluster.

Fluxo típico:

- 1. Você escreve a lógica PySpark localmente
- 2. Roda docker compose up — sobe o ambiente completo
- 3. O pipeline processa os dados pelas camadas bronze → silver → gold
- 4. Você inspeciona o resultado no storage local
- 5. Validado, ajusta o código e sobe para o Databricks

Para simular o storage (S3/ADLS) localmente, use MinIO — um servidor de objetos compatível com S3 que roda num container Docker. Assim você replica o ambiente de nuvem localmente.

Docker em projetos web

Tipo de projeto	Usa Docker?	Observação
Backend / API (FastAPI, Flask, Node)	Sim	Principal caso de uso
Frontend Next.js (modo servidor)	Sim	Tem processo Node rodando
Frontend Next.js (export estático)	Não necessário	Arquivos servidos por CDN
App mobile (React Native, Kotlin)	Não	Roda no dispositivo — acessa hardware
App desktop (Electron, WPF)	Não	Roda no SO do usuário
Banco de dados local de dev	Sim	Subir Postgres, Redis etc. sem instalar

Boas práticas

- **Use imagens slim:** prefira python:3.12-slim ao invés de python:3.12. Menor, mais rápido, menos vulnerabilidades.
- **Nunca coloque segredos na imagem:** senhas e tokens sempre via variáveis de ambiente ou .env externo.
- **Um processo por container:** API num container, banco em outro. Facilita escalar e substituir partes.
- **Ordene as layers pelo que muda menos:** dependências antes do código — aproveita o cache.
- **Use .dockerignore:** evita copiar .git, __pycache__, .env e outros arquivos desnecessários para a imagem.

Docker + Git — o que versionar

Arquivo	Versionar?	Motivo
Dockerfile	Sim	Define o ambiente — essencial
docker-compose.yml	Sim	Define os serviços — essencial
.env.example	Sim	Chaves sem valores — documenta o que é necessário
.dockerignore	Sim	Evita lixo na imagem
.env	Não	Contém segredos — no .gitignore
Imagens geradas	Não	Ficam no registry (Docker Hub), não no Git

Com o Dockerfile versionado, qualquer pessoa que clonar o repositório consegue rodar o projeto com `docker compose up` — sem instalar nada manualmente na máquina. Esse é o objetivo final.

Dúvidas respondidas

Precisa saber linha a linha do Dockerfile?

Não. Na prática do mercado, ninguém escreve Dockerfile do zero toda vez. O fluxo real é: você tem um template base para cada tipo de projeto, copia e ajusta 2 ou 3 linhas (versão do Python, nome do arquivo principal). Entender o que cada instrução faz é importante — memorizar sintaxe não é necessário.

A config muda muito entre projeto de programação e de dados?

A estrutura do Dockerfile e do Compose é idêntica. O que cresce num projeto de dados são os serviços no Compose e o tamanho da imagem — principalmente por causa do Java, que o Spark exige. Um projeto Python simples gera imagem de ~200MB. Com Spark, fica em torno de 1.5GB.

Configurar Spark é mais fácil com Docker?

Comparado a instalar Spark na mão (Java + variáveis de ambiente + PATH), Docker é muito mais simples. Mas o atalho ainda melhor é usar uma imagem pronta:

```
# no docker-compose.yml
services:
  spark:
    image: bitnami/spark:3.5
```

A imagem bitnami/spark já vem com Java, Spark e variáveis de ambiente configuradas. Você não escreve nada disso no Dockerfile. Atenção: localmente o Spark roda em modo standalone (sem cluster) — suficiente para desenvolver e testar lógica.

Next.js usa Docker ou não?

Depende do modo de deploy. Se você usa App Router, Server Components ou API Routes, tem um servidor Node rodando continuamente — Docker se aplica perfeitamente. Se o projeto gera um export estático (HTML/CSS/JS puro), não tem processo rodando e você serve os arquivos por CDN. A maioria dos projetos Next.js modernos usa o modo servidor.

Posso criar minha própria imagem do zero?

Sim. O fluxo é: escreve o Dockerfile, roda docker build para construir a imagem localmente, e opcionalmente faz docker push para publicar no Docker Hub. Na prática você quase nunca começa do zero absoluto — você parte de uma imagem base com FROM e constrói em cima dela:

FROM	O que já vem pronto
python:3.12	Python instalado — você adiciona suas libs e código
python:3.12-slim	Python sem pacotes extras — imagem mais leve

bitnami/spark:3.5	Java + Spark configurados — você adiciona sua lógica
ubuntu:22.04	SO base — você instala tudo manualmente (raramente necessário)